Week 11: Censored Models and Sample Selection

Samuel Rowe adapted from Wooldridge (2011), Mitchell (2020), and UCLA OARC (2025)

2025-11-19

Contents


```
bookdown::render_book()
```

Overview

This week we will be covering:

1. Censored Regression Models
2. Truncated Regression Models
3. Sample Selection
4. Subgroup Functions Part II

Chapter 1

Censored Models

1.1 Censored Regression Model - Right-Censoring

Lesson: We can use a Tobit estimator for right-censored model

We need to summarize the dependent variable to see the right-censored value. We know that weekly earnings in the Current Population Survey are top-coded or right-censored at $2884.61. This may bias our estimate, so we'll compare a OLS model and a Tobit model for right-censored data. Remember a Tobit estimator for a censored model is different from a corner solution.

```
use "/Users/Sam/Desktop/Econ 645/Data/CPS/jan2024.dta", clear
sum earnings if prerelg==1, detail
histogram earnings if prerelg==1, normal title(Histogram of Weekly Earnings) caption("Source: Cur
graph export "/Users/Sam/Desktop/Econ 645/R Markdown/week9_histogram_earnings.png", replace
```

```
                  Weekly Earnings: pternwa
-------------------------------------------------------------
      Percentiles      Smallest
 1%           70              0
 5%          225              0
10%          360              0       Obs              10,666
25%          656              0       Sum of Wgt.      10,666

50%         1000                      Mean           1230.474
                        Largest       Std. Dev.      788.1457
75%         1680        2884.61
90%       2692.3        2884.61       Variance       621173.7
```

```
95%      2884.61        2884.61       Skewness       .7779644
99%      2884.61        2884.61       Kurtosis       2.648218

(bin=40, start=0, width=72.11525)

(file /Users/Sam/Desktop/Econ 645/R Markdown/week9_histogram_earnings.png written in PN
```

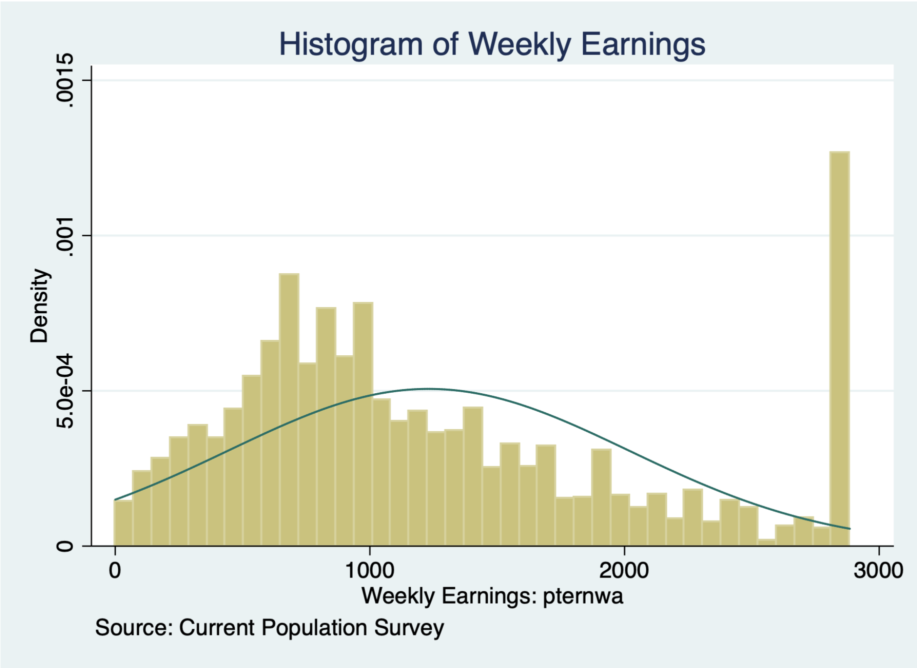


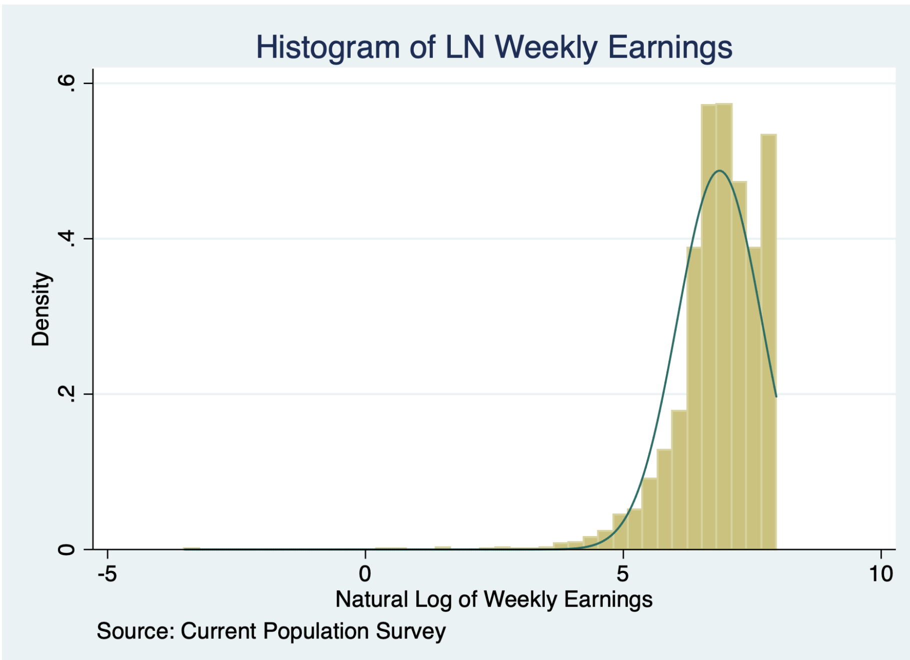
Figure 1.1: Histogram of Earnings

Let us take a look at the natural log of earnings

```
use "/Users/Sam/Desktop/Econ 645/Data/CPS/jan2024.dta", clear
sum lnearnings, detail
histogram lnearnings if prerelg==1, normal title(Histogram of LN Weekly Earnings) capti
graph export "/Users/Sam/Desktop/Econ 645/R Markdown/week9_histogram_lnearnings.png", r
```

```
               Natural Log of Weekly Earnings
-------------------------------------------------------------
      Percentiles      Smallest
 1%     4.356709      -3.506558
 5%     5.420535      -3.506558
10%     5.886104      -3.506558       Obs              10,652
25%      6.49224         .48858       Sum of Wgt.      10,652
```

```
50%      6.907755                       Mean           6.86502
                        Largest         Std. Dev.      .8181897
75%      7.426549       7.967145
90%      7.898151       7.967145        Variance       .6694343
95%      7.967145       7.967145        Skewness      -1.791008
99%      7.967145       7.967145        Kurtosis       13.92768

(bin=40, start=-3.5065579, width=.28684257)

(file /Users/Sam/Desktop/Econ 645/R Markdown/week9_histogram_lnearnings.png written in PNG format
```



We'll estimate the following Mincer Equation.

$$ln(wwages_i) = \beta_0 + \beta_1 edu_i + \beta_2 exp + \beta_3 exp^2 \beta_4 marital_i + \beta_5 veteran_i + \beta_6 union_i + \beta_7 female_i + \beta_8 race_i + u_i$$

We'll need to use the option, *ul(right-censored-value)* with our Tobit estimator.

```
sum lnearnings, detail
return list
local maxval `r(max)'
tobit lnearnings i.edu exp expsq i.marital i.veteran i.union i.female i.race, ul(`maxval')
```

```
                Natural Log of Weekly Earnings
```

```
-------------------------------------------------------------
      Percentiles      Smallest
 1%     4.356709      -3.506558
 5%     5.420535      -3.506558
10%     5.886104      -3.506558       Obs              10,652
25%      6.49224         .48858       Sum of Wgt.      10,652

50%     6.907755                      Mean            6.86502
                        Largest       Std. Dev.      .8181897
75%     7.426549       7.967145
90%     7.898151       7.967145       Variance       .6694343
95%     7.967145       7.967145       Skewness      -1.791008
99%     7.967145       7.967145       Kurtosis       13.92768


scalars:
                 r(N) =  10652
             r(sum_w) =  10652
              r(mean) =  6.865020483352663
               r(Var) =  .6694343494891623
                r(sd) =  .8181896781854207
          r(skewness) =  -1.791007993978133
          r(kurtosis) =  13.92768307319675
               r(sum) =  73126.19818867257
               r(min) =  -3.506557897319982
               r(max) =  7.967144987828557
                r(p1) =  4.356708826689592
                r(p5) =  5.420534999272286
               r(p10) =  5.886104031450156
               r(p25) =  6.492239835020471
               r(p50) =  6.907755278982137
               r(p75) =  7.426549072397305
               r(p90) =  7.898151125863075
               r(p95) =  7.967144987828557
               r(p99) =  7.967144987828557
```

```
Tobit regression                                Number of obs     =     10,568
                                                LR chi2(17)       =    3843.62
                                                Prob > chi2       =     0.0000
Log likelihood = -11364.619                     Pseudo R2         =     0.1446

------------------------------------------------------------------------------------
                             lnearnings |      Coef.   Std. Err.      t    P>|t|     [9
----------------------------------------+-------------------------------------------
```

```
                                     edu |
                                  HS/GED  |   .3303562   .0299199    11.04   0.000     .2717075
                                      AA  |    .444737   .0351051    12.67   0.000     .3759243
                                   BS/BA  |    .831706   .0318545    26.11   0.000     .7692652
                                AdDegree  |   1.046186    .034142    30.64   0.000     .9792607
                                          |
                                     exp  |   .0575487    .002085    27.60   0.000     .0534617
                                   expsq  |  -.0009113   .0000337   -27.03   0.000    -.0009774   -
                                          |
                                 marital  |
             Divorced/Separated/Widowed   |  -.0461827   .0217659    -2.12   0.034    -.0888481   -
                          Never Married   |   -.129934   .0189855    -6.84   0.000    -.1671492   -
                                          |
                                 veteran  |
                                 Veteran  |    .042753   .0333981     1.28   0.201    -.0227135
                                          |
                                   union  |
                                   Union  |   .0448454   .0236468     1.90   0.058    -.0015067
                                          |
                                  female  |
                                  Female  |  -.3347149   .0143948   -23.25   0.000    -.3629314   -
                                          |
                          race_ethnicity  |
                                NH Asian  |   .0550537   .0846217     0.65   0.515    -.1108209
                                NH Black  |  -.0576148   .0829774    -0.69   0.487    -.2202662
 NH Native Hawaiian or Pacific Islander   |   .1314362   .1374275     0.96   0.339    -.1379477
                      Latino/a or Hispanic  |   .0166878   .0818777     0.20   0.839     -.143808
                          NH Multiracial  |   .0902634   .0954831     0.95   0.345    -.0969015
                                NH White  |   .0553029   .0803465     0.69   0.491    -.1021914
                                          |
                                   _cons  |   5.817179   .0897475    64.82   0.000     5.641257
------------------------------------------+--------------------------------------------------------
                                  /sigma  |   .7138391   .0052104                      .7036258
---------------------------------------------------------------------------------------------------
            0  left-censored observations
        9,690     uncensored observations
          878 right-censored observations at lnearnings >= 7.967145
```

We will compare OLS to the Censored Regression Model

```
est clear
eststo OLS: quietly reg lnearnings i.edu exp expsq i.marital i.veteran i.union i.female i.race
eststo Tobit: quietly tobit lnearnings i.edu exp expsq i.marital i.veteran i.union i.female i.rac

esttab OLS Tobit, drop(0.* 1.race* 1.mar* 1.edu) mtitle("OLS" "Tobit")
```

	(1) OLS	(2) Tobit
main		
2.edu	0.330***	0.330***
	(11.77)	(11.04)
3.edu	0.442***	0.445***
	(13.48)	(12.67)
4.edu	0.788***	0.832***
	(26.50)	(26.11)
5.edu	0.951***	1.046***
	(30.01)	(30.64)
exp	0.0558***	0.0575***
	(28.73)	(27.60)
expsq	-0.000887***	-0.000911***
	(-28.27)	(-27.03)
2.marital	-0.0389	-0.0462*
	(-1.92)	(-2.12)
3.marital	-0.118***	-0.130***
	(-6.66)	(-6.84)
1.veteran	0.0412	0.0428
	(1.34)	(1.28)
1.union	0.0670**	0.0448
	(3.05)	(1.90)
1.female	-0.304***	-0.335***
	(-22.81)	(-23.25)
2.race_eth~y	0.0325	0.0551
	(0.41)	(0.65)
3.race_eth~y	-0.0466	-0.0576
	(-0.60)	(-0.69)
4.race_eth~y	0.135	0.131
	(1.05)	(0.96)

```
5.race_eth~y       0.0232          0.0167
                   (0.30)          (0.20)

6.race_eth~y       0.0730          0.0903
                   (0.82)          (0.95)

7.race_eth~y       0.0549          0.0553
                   (0.73)          (0.69)

_cons               5.813***        5.817***
                  (69.47)         (64.82)
--------------------------------------------
sigma
_cons                               0.714***
                                 (137.00)
--------------------------------------------
N                   10568           10568
--------------------------------------------
t statistics in parentheses
* p<0.05, ** p<0.01, *** p<0.001
```

Interpretation A very similar interpretation to OLS, but we need normality and homoskedasticity for unbiased estimator. We can use a log-linear interpretation without a scaling factor, so the returns to education would only require $(e^{\beta} - 1) * 100$ interpretation.

1.2 Censored Regression Model - Duration analysis

Lesson: We can look at a censored data for duration analysis similar to Wooldridge's example.

This differs from a top-coded data, which we can use a Tobit analysis that we just saw.

We can look at the duration of time in months between an arrests for inmates in a North Carolina prison after being released from prison. We want to evaluate a work program to see if it is effective in increasing duration before recidivism occurs.

Note: 893 inmates have not been arrested during the period they were followed These observations are censored. The censoring times differed among inmates ranging from 70 to 81 months.

Our dependent variable duration (time in months) is transformed by natural logarithm. We have a bunch of observations that recidivate between 70 and 81 months.

```
use "/Users/Sam/Desktop/Econ 645/Data/Wooldridge/recid.dta", clear
tab durat
```

We have a bunch of observations that are censored after 69 months (but not all)

```
use "/Users/Sam/Desktop/Econ 645/Data/Wooldridge/recid.dta", clear
tab durat cens
```

```
           |         cens
     durat |         0          1 |     Total
-----------+----------------------+----------
         1 |         8          0 |         8
         2 |        15          0 |        15
         3 |        14          0 |        14
         4 |        13          0 |        13
         5 |        16          0 |        16
         6 |        18          0 |        18
         7 |        18          0 |        18
         8 |        16          0 |        16
         9 |        18          0 |        18
        10 |        22          0 |        22
        11 |        11          0 |        11
        12 |        14          0 |        14
        13 |        15          0 |        15
        14 |        16          0 |        16
        15 |        23          0 |        23
        16 |        11          0 |        11
        17 |         9          0 |         9
        18 |        16          0 |        16
        19 |         9          0 |         9
        20 |         8          0 |         8
        21 |        13          0 |        13
        22 |         7          0 |         7
        23 |        16          0 |        16
        24 |        12          0 |        12
        25 |        13          0 |        13
        26 |         8          0 |         8
        27 |        11          0 |        11
        28 |         9          0 |         9
        29 |         8          0 |         8
        30 |         7          0 |         7
        31 |         6          0 |         6
```

32	6	0	6
33	6	0	6
34	4	0	4
35	5	0	5
36	6	0	6
37	6	0	6
38	4	0	4
39	4	0	4
40	2	0	2
41	7	0	7
42	5	0	5
43	5	0	5
44	4	0	4
45	4	0	4
46	7	0	7
47	4	0	4
48	1	0	1
49	4	0	4
50	5	0	5
51	2	0	2
52	2	0	2
53	8	0	8
54	3	0	3
55	5	0	5
56	2	0	2
57	4	0	4
58	1	0	1
59	5	0	5
60	3	0	3
62	3	0	3
63	2	0	2
64	1	0	1
65	2	0	2
66	3	0	3
67	3	0	3
68	4	0	4
69	2	0	2
70	2	103	105
71	2	88	90
72	1	84	85
73	1	107	108
74	1	71	72
75	0	44	44
76	0	105	105
77	1	60	61
78	0	54	54

```
        79 |         0         60 |        60
        80 |         0         69 |        69
        81 |         0         48 |        48
-----------+----------------------+----------
     Total |       552        893 |     1,445
```

We'll use the *stset* command and set failure at cens==0. We'll use the *stset* command to time set survival.

```
use "/Users/Sam/Desktop/Econ 645/Data/Wooldridge/recid.dta", clear
stset ldurat, failure(cens==0)
```

```
     failure event:  cens == 0
obs. time interval:  (0, ldurat]
 exit on or before:  failure

------------------------------------------------------------------------------
       1445  total observations
          8  observations end on or before enter()
------------------------------------------------------------------------------
       1437  observations remaining, representing
        544  failures in single-record/single-failure data
   5411.742  total analysis time at risk and under observation
                                                at risk from t =         0
                                     earliest observed entry t =         0
                                          last observed exit t =  4.394449
```

We'll use the *streg* command and set the distribution

$$ldurat_i = \alpha + = \delta workprg_i + \beta_2 tserved_i + \beta_3 felon_i + \beta_4 alcohol_i + \beta_5 drugs_i + \beta_6 educ_i + x_i'\gamma + \varepsilon_i$$

Where x' are demographics of race, marital status, and age.

```
streg workprg priors tserved felon alcohol drugs black married educ age, dist(weibull)
```

```
     failure event:  cens == 0
obs. time interval:  (0, ldurat]
 exit on or before:  failure

------------------------------------------------------------------------------
       1445  total observations
          8  observations end on or before enter()
------------------------------------------------------------------------------
```

```
      1437  observations remaining, representing
       544  failures in single-record/single-failure data
  5411.742  total analysis time at risk and under observation
                                             at risk from t =         0
                                  earliest observed entry t =         0
                                       last observed exit t =  4.394449

         failure _d:  cens == 0
   analysis time _t:  ldurat

Fitting constant-only model:

Iteration 0:   log likelihood = -1254.5107
Iteration 1:   log likelihood = -1100.1536
Iteration 2:   log likelihood = -1079.1128
Iteration 3:   log likelihood = -1078.7957
Iteration 4:   log likelihood = -1078.7957

Fitting full model:

Iteration 0:   log likelihood = -1078.7957
Iteration 1:   log likelihood = -1034.1821
Iteration 2:   log likelihood = -1001.9186
Iteration 3:   log likelihood = -1000.5996
Iteration 4:   log likelihood = -1000.5919
Iteration 5:   log likelihood = -1000.5919

Weibull regression -- log relative-hazard form

No. of subjects =        1,437                  Number of obs    =      1,437
No. of failures =          544
Time at risk    =  5411.742317
                                                LR chi2(10)      =     156.41
Log likelihood  =   -1000.5919                  Prob > chi2      =     0.0000

------------------------------------------------------------------------------
          _t |      Coef.   Std. Err.      z    P>|z|     [95% Conf. Interval]
-------------+----------------------------------------------------------------
     workprg |   .0780722    .091463     0.85   0.393    -.1011921    .2573364
      priors |   .0834203   .0139155     5.99   0.000     .0561465    .1106941
     tserved |   .0134545   .0016851     7.98   0.000     .0101519    .0167572
       felon |   -.287139    .106697    -2.69   0.007    -.4962614   -.0780167
     alcohol |   .4399456     .10658     4.13   0.000     .2310527    .6488385
       drugs |   .2920932   .0983695     2.97   0.003     .0992926    .4848938
       black |   .4515388   .0889883     5.07   0.000     .2771249    .6259527
     married |   -.146192   .1098131    -1.33   0.183    -.3614216    .0690377
```

```
        educ |   -.023948    .019578    -1.22   0.221    -.0623202    .0144241
         age |  -.0036431   .0005284    -6.89   0.000    -.0046788   -.0026073
       _cons |  -3.639277   .3077568   -11.83   0.000     -4.24247   -3.036085
-------------+----------------------------------------------------------------
       /ln_p |   .9214587   .0396737    23.23   0.000     .8436997    .9992178
-------------+----------------------------------------------------------------
           p |   2.512953   .0996982                      2.324953    2.716156
         1/p |   .3979381   .0157877                      .3681673    .4301163
------------------------------------------------------------------------------
```

Interpretation: Given the log-linear function form, we can easily determine the estimated percent change in duration before criminal recidivism.

```
display (exp(.0780722)-1)*100
```

```
8.1200719
```

Being a part of the work program increase the duration of time before recidivism, but it is not statistically significant.

```
display (exp(-.287139)-1)*100
```

```
-24.959259
```

Being a felon reduces the duration of time before recidivism, where a felon has as 24% decrease in duration of time before recidivism.

Chapter 2

Truncated Regression Models

We will again look at the Janurary 2024 Current Population Survey data. However, we will truncate the data at \$2884 per week by dropping all observation at or above this threshold.

2.1 Truncated Data

We will set the threshold $c_i \geq 2884$. Every observation at or above this threshold will be dropped.

```
use "/Users/Sam/Desktop/Econ 645/Data/CPS/jan2024.dta", clear
sum earnings if prerelg==1, detail
est clear
eststo OLS: quietly reg lnearnings i.edu exp expsq i.marital i.veteran i.union i.female i.race if
drop if earnings >= 2884
sum earnings if prerelg==1, detail
```

```
                  Weekly Earnings: pternwa
-------------------------------------------------------------
      Percentiles      Smallest
 1%           70              0
 5%          225              0
10%          360              0       Obs              10,666
25%          656              0       Sum of Wgt.      10,666

50%         1000                      Mean           1230.474
```

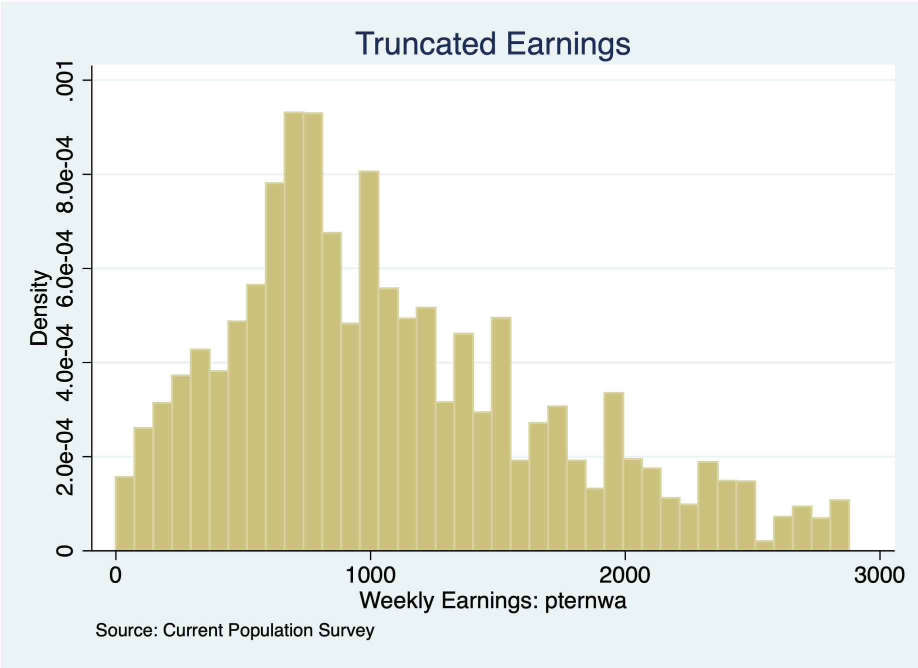
```
                         Largest       Std. Dev.      788.1457
75%          1680        2884.61
90%        2692.3        2884.61       Variance       621173.7
95%       2884.61        2884.61       Skewness       .7779644
99%       2884.61        2884.61       Kurtosis       2.648218
```

```
(28,566 observations deleted)

                  Weekly Earnings: pternwa
-------------------------------------------------------------
      Percentiles      Smallest
 1%          62.5             0
 5%           206             0
10%           336             0       Obs                9,767
25%           620             0       Sum of Wgt.        9,767

50%           960                     Mean            1078.221
                        Largest       Std. Dev.       635.0599
75%          1450          2880
90%          2000          2880       Variance        403301.1
95%          2320          2880       Skewness        .7302792
99%          2780          2880       Kurtosis        2.975473
```

Our largest value is now $2880 weekly earnings, which is just below the threshold.

Let's look at a histogram of the truncated weekly earnings.

```
histogram earnings if prerelg, title("Truncated Earnings") note("Source: Current Popula
graph export "/Users/Sam/Desktop/Econ 645/Data/CPS/jan2024_trunc.png", replace
```



We no longer have a spike in density at 2884 like we did with the censored data.

2.2 Truncated Regression Models

Next, we will use the command *truncreg* and set the option *ul()* at 2884.

```
eststo Truncated: truncreg lnearnings i.edu exp expsq i.marital i.veteran i.union i.female i.race
```

```
(note: 0 obs. truncated)

Fitting full model:

Iteration 0:   log likelihood = -9654.7635
Iteration 1:   log likelihood = -9654.7551
Iteration 2:   log likelihood = -9654.7551

Truncated regression
Limit:   lower =       -inf                    Number of obs =    =      9,669
         upper =       2884                    Wald chi2(17)      =    3276.34
Log likelihood = -9654.7551                    Prob > chi2        =     0.0000

----------------------------------------------------------------------------------------------------------
                             lnearnings |      Coef.   Std. Err.      z    P>|z|     [95% Conf. I
```

```
---------------------------------------+----------------------------------------------
                                   edu |
                                HS/GED  |   .3230896   .0276751    11.67   0.000     .2
                                    AA  |   .4320131   .0325527    13.27   0.000     .3
                                 BS/BA  |   .6971765   .0297669    23.42   0.000     .6
                              AdDegree  |   .8088922   .0325213    24.87   0.000     .7
                                       |
                                   exp |    .054815   .0019609    27.95   0.000     .0
                                 expsq |  -.0008866   .0000319   -27.81   0.000    -.0
                                       |
                               marital |
           Divorced/Separated/Widowed  |  -.0290168   .0206418    -1.41   0.160    -.0
                        Never Married  |  -.1028859   .0178986    -5.75   0.000    -.1
                                       |
                               veteran |
                               Veteran  |   .0280901   .0322307     0.87   0.383    -.0
                                       |
                                 union |
                                 Union  |   .1126058   .0223161     5.05   0.000
                                       |
                                female |
                                Female  |  -.2718903   .0137518   -19.77   0.000    -.2
                                       |
                        race_ethnicity |
                              NH Asian  |  -.0426197   .0795044    -0.54   0.592    -.1
                              NH Black  |    -.04673   .0774491    -0.60   0.546    -.1
NH Native Hawaiian or Pacific Islander  |    .136987   .1278521     1.07   0.284    -.1
                   Latino/a or Hispanic  |   .0151638    .076424     0.20   0.843    -.1
                         NH Multiracial  |   .0275207   .0898433     0.31   0.759     -
                               NH White  |    .035842   .0750013     0.48   0.633    -.1
                                       |
                                 _cons |   5.814378   .0837643    69.41   0.000     5
---------------------------------------+----------------------------------------------
                                /sigma |   .6567763   .0047229   139.06   0.000     .6
--------------------------------------------------------------------------------------
```

We can directly compare our OLS and Truncated Regression Model coefficients.

```
esttab OLS Truncated
```

```
                      (1)             (2)
                      OLS             TRM
--------------------------------------------
main
1.edu                   0               0
```

	(.)	(.)
2.edu	0.330***	0.323***
	(11.77)	(11.67)
3.edu	0.442***	0.432***
	(13.48)	(13.27)
4.edu	0.788***	0.697***
	(26.50)	(23.42)
5.edu	0.951***	0.809***
	(30.01)	(24.87)
exp	0.0558***	0.0548***
	(28.73)	(27.95)
expsq	-0.000887***	-0.000887***
	(-28.27)	(-27.81)
1.marital	0	0
	(.)	(.)
2.marital	-0.0389	-0.0290
	(-1.92)	(-1.41)
3.marital	-0.118***	-0.103***
	(-6.66)	(-5.75)
0.veteran	0	0
	(.)	(.)
1.veteran	0.0412	0.0281
	(1.34)	(0.87)
0.union	0	0
	(.)	(.)
1.union	0.0670**	0.113***
	(3.05)	(5.05)
0.female	0	0
	(.)	(.)
1.female	-0.304***	-0.272***
	(-22.81)	(-19.77)

```
1.race_eth~y            0               0
                      (.)             (.)

2.race_eth~y       0.0325         -0.0426
                   (0.41)         (-0.54)

3.race_eth~y      -0.0466         -0.0467
                  (-0.60)         (-0.60)

4.race_eth~y        0.135           0.137
                   (1.05)          (1.07)

5.race_eth~y       0.0232          0.0152
                   (0.30)          (0.20)

6.race_eth~y       0.0730          0.0275
                   (0.82)          (0.31)

7.race_eth~y       0.0549          0.0358
                   (0.73)          (0.48)

_cons               5.813***        5.814***
                  (69.47)         (69.41)
--------------------------------------------
sigma
_cons                               0.657***
                                 (139.06)
--------------------------------------------
N                   10568            9669
--------------------------------------------
t statistics in parentheses
* p<0.05, ** p<0.01, *** p<0.001
```

Interpretation: Our union wage premium is estimated to be $((e^{0.067}) - 1) * 100\% = 6.9\%$ with the OLS estimator, while our union wage premium is estimated to be $((e^{0.113}) - 1) * 100\% = 12.0\%$.

We can see that our OLS estimates are biased upwards for education and experience, but downward biased for union wage premium.

Source: https://stats.oarc.ucla.edu/stata/output/truncated-regression/

Chapter 3

Sample Selection Correction

Lesson: Similar to tobit, when we don't account for the truncation of the data, or why certain parts of the population are not sampled, we will commit a type of omitted variable bias without our lambda(zy)-hat. We can use a Heckit method for sample selection correction.

We want to see if there is sample selection bias due to unobservable wage offers for non-working women.

1. We need to estimate a logit or probit to test and correct for sample selection bias due to unobserved wage offer for nonworking women. Spousal income, education, experience, age, and number of kids less than 6.

   $$ln(wages_i) = \beta_0 + \mathbf{x}' \ + u_i$$

   Where $\mathbf{x}$ is a vector that includes education, experience, and experience squared

2. We use the Heckit command to implement a Heckman Method for sample selection correction.

$$ln(wages_i) = \beta_0 + \mathbf{x}' \ + \mathbf{z}' \ + u_i$$

Where $\mathbf{z}$ is a vector that includes spousal income, education, experience, age, and number of kids less than 6.

3.1 OLS

```
use "/Users/Sam/Desktop/Econ 645/Data/Wooldridge/mroz.dta", clear
reg lwage educ exper expersq
```

```
      Source |       SS           df       MS      Number of obs   =       428
-------------+----------------------------------   F(3, 424)       =     26.29
       Model |  35.0222967         3  11.6740989   Prob > F        =    0.0000
    Residual |  188.305144       424  .444115906   R-squared       =    0.1568
-------------+----------------------------------   Adj R-squared   =    0.1509
       Total |  223.327441       427  .523015084   Root MSE        =    .66642

------------------------------------------------------------------------------
       lwage |      Coef.   Std. Err.      t    P>|t|     [95% Conf. Interval]
-------------+----------------------------------------------------------------
        educ |   .1074896   .0141465     7.60   0.000     .0796837    .1352956
       exper |   .0415665   .0131752     3.15   0.002     .0156697    .0674633
     expersq |  -.0008112   .0003932    -2.06   0.040    -.0015841   -.0000382
       _cons |  -.5220406   .1986321    -2.63   0.009    -.9124667   -.1316144
------------------------------------------------------------------------------
```

Notice: an assumption is used to exclude spousal income, age, kids less than 6, and kids greater than 6 from our main regression.

```
display (exp(_b[educ])-1)*100
```

```
11.347933
```

3.2 Heckman Method - Heckit

We will use a subset of all exogenous variable 1. What are the factors correlated with being in the labor force? 2. What is the impact of education and experience on wages

```
use "/Users/Sam/Desktop/Econ 645/Data/Wooldridge/mroz.dta", clear
heckman lwage educ exper expersq, select(inlf=nwifeinc educ exper expersq age kidslt6 k
```

```
Heckman selection model -- two-step estimates   Number of obs     =        753
(regression model with sample selection)        Censored obs      =        325
                                                Uncensored obs    =        428
```

```
                                                Wald chi2(3)      =      51.53
                                                Prob > chi2       =     0.0000

------------------------------------------------------------------------------
             |      Coef.   Std. Err.      z    P>|z|     [95% Conf. Interval]
-------------+----------------------------------------------------------------
lwage        |
        educ |   .1090655    .015523     7.03   0.000     .0786411      .13949
       exper |   .0438873   .0162611     2.70   0.007     .0120163    .0757584
     expersq |  -.0008591   .0004389    -1.96   0.050    -.0017194    1.15e-06
       _cons |  -.5781032   .3050062    -1.90   0.058    -1.175904     .019698
-------------+----------------------------------------------------------------
inlf         |
    nwifeinc |  -.0120237   .0048398    -2.48   0.013    -.0215096   -.0025378
        educ |   .1309047   .0252542     5.18   0.000     .0814074     .180402
       exper |   .1233476   .0187164     6.59   0.000     .0866641    .1600311
     expersq |  -.0018871      .0006    -3.15   0.002     -.003063   -.0007111
         age |  -.0528527   .0084772    -6.23   0.000    -.0694678   -.0362376
     kidslt6 |  -.8683285   .1185223    -7.33   0.000    -1.100628    -.636029
     kidsge6 |    .036005   .0434768     0.83   0.408     -.049208    .1212179
       _cons |   .2700768    .508593     0.53   0.595    -.7267473    1.266901
-------------+----------------------------------------------------------------
mills        |
      lambda |   .0322619   .1336246     0.24   0.809    -.2296376    .2941613
-------------+----------------------------------------------------------------
         rho |    0.04861
       sigma |  .66362875
------------------------------------------------------------------------------
```

```
est clear
eststo OLS: reg lwage educ exper expersq
eststo Heckman: heckman lwage educ exper expersq, select(inlf=nwifeinc educ exper expersq age kid
esttab, mtitle
```

```
                      (1)             (2)
                      OLS         Heckman
--------------------------------------------
main
educ                0.107***        0.109***
                   (7.60)          (7.03)

exper              0.0416**        0.0439**
                   (3.15)          (2.70)

expersq         -0.000811*      -0.000859
```

```
                  (-2.06)         (-1.96)

_cons              -0.522**        -0.578
                  (-2.63)         (-1.90)
--------------------------------------------
inlf
nwifeinc                          -0.0120*
                                  (-2.48)

educ                                0.131***
                                   (5.18)

exper                               0.123***
                                   (6.59)

expersq                         -0.00189**
                                  (-3.15)

age                               -0.0529***
                                  (-6.23)

kidslt6                            -0.868***
                                  (-7.33)

kidsge6                            0.0360
                                   (0.83)

_cons                               0.270
                                   (0.53)
--------------------------------------------
mills
lambda                             0.0323
                                   (0.24)
--------------------------------------------
N                      428            753
--------------------------------------------
t statistics in parentheses
* p<0.05, ** p<0.01, *** p<0.001
```

There is no real evidence of sample selection bias in the wage offer equation. Our lambda-hat is not statistically significant and we fail to reject $H_0 : \rho = 0$. Also, we notice very little difference between our OLS and Heckman Method.

Interpretation: we will have a similar interpretation to our OLS. We have a log-linear model so our interpretation of the return would be e^{β} or $(e^{\beta} - 1) * 100$. Our $\hat{\lambda}$ would be a potential omitted variable, but our example shows that we fail to reject the null hypothesis and selection bias is does not appear to be problematic.

Chapter 4

Subgroup Functions Part II

4.1 Computing values within subgroups: Running sums

As we mentioned earlier, when we use *egen sum* vs *gen sum*, we get different results. *Egen sum()* is similar to *egen total()*, but it can be confusing. When we use *gen* with *sum()*, we generate a **RUNNING** sum not a constant of total.

We can generate the tv running sum across all individuals over time

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use tv1, clear
generate tvrunsum = sum(tv)
```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell
```

We can generate the tv running sum within an individuals time period

```
bysort kidid (dt): generate bytvrunsum=sum(tv)
```

We can generate the total sum with an individuals time period

```
bysort kidid (dt): egen bytvsum=total(tv)
```

We can generate the total sum for all individuals over time

```
egen tvsum = total(tv)
```

We can also calculate a running average

```
bysort kidid (dt): generate bytvrunavg=sum(tv)/_n
```

We can compute the individual's average average

```
bysort kidid (dt): egen bytvavg = mean(tv)
list kidid tv tv* by*, sepby(kidid)
```

```
     | kidid   tv   tv   tvrunsum   tvsum   bytvru~m   bytvsum   bytvrun~g   bytvavg |
     |----------------------------------------------------------------------------|
  1. |     1    1    1          1      38          1         4           1         2 |
  2. |     1    3    3          4      38          4         4           2         2 |
     |----------------------------------------------------------------------------|
  3. |     2    8    8         12      38          8         8           8         8 |
     |----------------------------------------------------------------------------|
  4. |     3    2    2         14      38          2        14           2       3.5 |
  5. |     3    5    5         19      38          7        14         3.5       3.5 |
  6. |     3    1    1         20      38          8        14   2.6666667       3.5 |
  7. |     3    6    6         26      38         14        14         3.5       3.5 |
     |----------------------------------------------------------------------------|
  8. |     4    7    7         33      38          7        12           7         4 |
  9. |     4    1    1         34      38          8        12           4         4 |
 10. |     4    4    4         38      38         12        12           4         4 |
     +----------------------------------------------------------------------------+
```

4.2 Computing values within subgroups: More examples

There are other useful calculations we can do with subscripting/indexing. Some do overlap with egen, but it is helpful to know the differences.

Counting

Count the number of observations: this can be done with subscripting or egen depending upon what we want

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use tv1, clear

*Generate total observation count per individual missing or not missing:
bysort kidid (dt): generate idcount=_N

*Generate the total observation without missing
bysort kidid (dt): egen idcount_nomiss = count(tv)

*Generate a running count of an observation
bysort kidid (dt): gen idruncount = _n
list, sepby(kidid)
```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell

     +-----------------------------------------------------------------------------+
     | kidid          dt   female   wt   tv   vac   idcount   idcoun~s   idrunc~t |
     |-----------------------------------------------------------------------------|
  1. |     1   07jan2002        1   53    1     1         2          2          1 |
  2. |     1   08jan2002        1   55    3     1         2          2          2 |
     |-----------------------------------------------------------------------------|
  3. |     2   16jan2002        1   58    8     1         1          1          1 |
     |-----------------------------------------------------------------------------|
  4. |     3   18jan2002        0   60    2     0         4          4          1 |
  5. |     3   19jan2002        0   63    5     1         4          4          2 |
  6. |     3   21jan2002        0   66    1     1         4          4          3 |
  7. |     3   22jan2002        0   64    6     0         4          4          4 |
     |-----------------------------------------------------------------------------|
  8. |     4   10jan2002        1   62    7     0         3          3          1 |
  9. |     4   11jan2002        1   58    1     0         3          3          2 |
 10. |     4   13jan2002        1   55    4     0         3          3          3 |
     +-----------------------------------------------------------------------------+
```

Generate Binaries

We can generate binary variables to find first and last observations or nth observation. This differences from id counts, we are generating binaries for when the qualifier is true.

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use tv1, clear

*Find individuals with only one observation
bysort kidid (dt): generate singleob = (_N==1)
```

```
*Find the first observation of an individual
bysort kidid (dt): generate firstob = (_n==1)

*Find the last observation of an individual
bysort kidid (dt): generate lastob = (_n==_N)
list, sepby(kidid)
```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell

     +------------------------------------------------------------------------------+
     | kidid          dt   female   wt   tv   vac   singleob   firstob   lastob |
     |------------------------------------------------------------------------------|
  1. |     1   07jan2002        1   53    1     1          0         1        0 |
  2. |     1   08jan2002        1   55    3     1          0         0        1 |
     |------------------------------------------------------------------------------|
  3. |     2   16jan2002        1   58    8     1          1         1        1 |
     |------------------------------------------------------------------------------|
  4. |     3   18jan2002        0   60    2     0          0         1        0 |
  5. |     3   19jan2002        0   63    5     1          0         0        0 |
  6. |     3   21jan2002        0   66    1     1          0         0        0 |
  7. |     3   22jan2002        0   64    6     0          0         0        1 |
     |------------------------------------------------------------------------------|
  8. |     4   10jan2002        1   62    7     0          0         1        0 |
  9. |     4   11jan2002        1   58    1     0          0         0        0 |
 10. |     4   13jan2002        1   55    4     0          0         0        1 |
     +------------------------------------------------------------------------------+
```

We can create binaries depending upon leads and lags too. Look for a change in vac

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use tv1, clear
bysort kidid (dt): generate vacstart=(vac==1) & (vac[_n-1]==0)
bysort kidid (dt): generate vacend=(vac==1) & (vac[_n+1]==0)
list kidid dt vac*, sepby(kidid)
```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell

     +----------------------------------------------+
     | kidid          dt   vac   vacstart   vacend |
     |----------------------------------------------|
  1. |     1   07jan2002     1          0        0 |
  2. |     1   08jan2002     1          0        0 |
```

```
     |-------------------------------------------------|
  3. |      2   16jan2002     1          0         0 |
     |-------------------------------------------------|
  4. |      3   18jan2002     0          0         0 |
  5. |      3   19jan2002     1          1         0 |
  6. |      3   21jan2002     1          0         1 |
  7. |      3   22jan2002     0          0         0 |
     |-------------------------------------------------|
  8. |      4   10jan2002     0          0         0 |
  9. |      4   11jan2002     0          0         0 |
 10. |      4   13jan2002     0          0         0 |
     +-------------------------------------------------+
```

Fill in Missing

Another useful tool that we should use with caution is filling in missings. This should only really be applied when we have a constant variable that does not change over time.

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use tv2, clear
sort kidid dt
list, sepby(kidid)
```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell

     +------------------------------------------------+
     | kidid          dt   female   wt   tv   vac |
     |------------------------------------------------|
  1. |      1   07jan2002        1   53    1     1 |
  2. |      1   08jan2002        1   55    3     1 |
     |------------------------------------------------|
  3. |      2   16jan2002        1   58    8     1 |
     |------------------------------------------------|
  4. |      3   18jan2002        0   60    2     0 |
  5. |      3   19jan2002        0    .    .     . |
  6. |      3   21jan2002        0   66    .     1 |
  7. |      3   22jan2002        0   64    6     0 |
     |------------------------------------------------|
  8. |      4   10jan2002        1   62    7     0 |
  9. |      4   11jan2002        1   58    .     . |
 10. |      4   13jan2002        1    .    4     0 |
     +------------------------------------------------+
```

We can backfill the observation with the last nonmissing observation. First generate a copy of the variable with missing values.

```
generate tvimp1 = tv
bysort kidid (dt): replace tvimp1 = tv[_n-1] if missing(tv)
list kidid dt tv tvimp1, sepby(kidid)
```

```
(3 missing values generated)

(2 real changes made)

     +---------------------------------+
     | kidid          dt   tv   tvimp1 |
     |---------------------------------|
  1. |     1   07jan2002    1        1 |
  2. |     1   08jan2002    3        3 |
     |---------------------------------|
  3. |     2   16jan2002    8        8 |
     |---------------------------------|
  4. |     3   18jan2002    2        2 |
  5. |     3   19jan2002    .        2 |
  6. |     3   21jan2002    .        . |
  7. |     3   22jan2002    6        6 |
     |---------------------------------|
  8. |     4   10jan2002    7        7 |
  9. |     4   11jan2002    .        7 |
 10. |     4   13jan2002    4        4 |
     +---------------------------------+
```

Notice that we are still missing the 3rd observation for the 3rd kid. It cannot backfill the 3rd observation from the second observation, since the second observation is missing. There are a couple of strategies to use We can generate a new variable like Mitchell.

```
generate tvimp2 = tvimp1
bysort kidid (dt): replace tvimp2 = tvimp2[_n-1] if missing(tvimp2)
list kidid tv tvimp*, sepby(kidid)
```

```
(1 missing value generated)

(1 real change made)

     +-----------------------------+
     | kidid   tv   tvimp1   tvimp2 |
```

```
     |------------------------------|
  1. |      1    1         1        1 |
  2. |      1    3         3        3 |
     |------------------------------|
  3. |      2    8         8        8 |
     |------------------------------|
  4. |      3    2         2        2 |
  5. |      3    .         2        2 |
  6. |      3    .         .        2 |
  7. |      3    6         6        6 |
     |------------------------------|
  8. |      4    7         7        7 |
  9. |      4    .         7        7 |
 10. |      4    4         4        4 |
     +------------------------------+
```

You can just replace tvimp1 twice instead of generating a new variable, but that is up to the user. You would use tv[_n-1] for the first replace and tvimp1[_n-1] for the second replace.

Interpolation

We may need to interpolate between 2 known values and assume a linear trend.

```
generate tvimp3=tv
*Interpolate for 1 missing value between two known values
bysort kidid (dt): replace tvimp3 = (tv[_n-1]+tv[_n+1])/2 if missing(tv)
list kidid tv tvimp3, sepby(kidid)
```

```
(3 missing values generated)

(1 real change made)

     +---------------------+
     | kidid   tv   tvimp3 |
     |---------------------|
  1. |     1    1        1 |
  2. |     1    3        3 |
     |---------------------|
  3. |     2    8        8 |
     |---------------------|
  4. |     3    2        2 |
  5. |     3    .        . |
  6. |     3    .        . |
```

```
  7. |     3    6         6 |
     |---------------------|
  8. |     4    7         7 |
  9. |     4    .       5.5 |
 10. |     4    4         4 |
     +---------------------+
```

This is a bit of hard coding, but you can interpolate with more than 1 missing.

```
bysort kidid (dt): replace tvimp3 = ((tvimp3[4]-tvimp3[1])/3)+tvimp3[_n-1] if missing(t
list kidid tv tvimp3, sepby(kidid)
```

```
(2 real changes made)

     +------------------------+
     | kidid   tv      tvimp3 |
     |------------------------|
  1. |     1    1           1 |
  2. |     1    3           3 |
     |------------------------|
  3. |     2    8           8 |
     |------------------------|
  4. |     3    2           2 |
  5. |     3    .   3.3333333 |
  6. |     3    .   4.6666667 |
  7. |     3    6           6 |
     |------------------------|
  8. |     4    7           7 |
  9. |     4    .         5.5 |
 10. |     4    4           4 |
     +------------------------+
```

Indicators

What is we want to find outliers in time-varying differences? We can generate indicators variables to find when a variable changes more than a set limit. For example we want to know if the tv viewing habits drop more than 2 hours

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use tv1, clear
bysort kidid (dt): generate tvchange = tv[_n]-tv[_n-1]
bysort kidid (dt): generate tvchangerate = ((tv[_n]-tv[_n-1])/tv[_n-1])
list, sepby(kidid)
```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell


(4 missing values generated)

(4 missing values generated)

     +-----------------------------------------------------------------+
     | kidid          dt   female   wt   tv   vac   tvchange   tvchange~e |
     |-----------------------------------------------------------------|
  1. |     1   07jan2002        1   53    1     1          .            . |
  2. |     1   08jan2002        1   55    3     1          2            2 |
     |-----------------------------------------------------------------|
  3. |     2   16jan2002        1   58    8     1          .            . |
     |-----------------------------------------------------------------|
  4. |     3   18jan2002        0   60    2     0          .            . |
  5. |     3   19jan2002        0   63    5     1          3          1.5 |
  6. |     3   21jan2002        0   66    1     1         -4          -.8 |
  7. |     3   22jan2002        0   64    6     0          5            5 |
     |-----------------------------------------------------------------|
  8. |     4   10jan2002        1   62    7     0          .            . |
  9. |     4   11jan2002        1   58    1     0         -6   -.85714286 |
 10. |     4   13jan2002        1   55    4     0          3            3 |
     +-----------------------------------------------------------------+
```

Generate an indicator variable to see if tvchange is less than -2. This is not very helpful with small datasets, but it is important with larger datasets such as the CPS.

```
gen tvchangeid=(tvchange <= -2) if !missing(tvchange)
list, sepby(kidid)
```

```
(4 missing values generated)

     +----------------------------------------------------------------------------+
     | kidid          dt   female   wt   tv   vac   tvchange   tvchange~e   tvchan~d |
     |----------------------------------------------------------------------------|
  1. |     1   07jan2002        1   53    1     1          .            .          . |
  2. |     1   08jan2002        1   55    3     1          2            2          0 |
     |----------------------------------------------------------------------------|
  3. |     2   16jan2002        1   58    8     1          .            .          . |
     |----------------------------------------------------------------------------|
  4. |     3   18jan2002        0   60    2     0          .            .          . |
  5. |     3   19jan2002        0   63    5     1          3          1.5          0 |
  6. |     3   21jan2002        0   66    1     1         -4          -.8          1 |
```

```
  7. |     3   22jan2002        0   64    6     0          5             5          0 |
     |----------------------------------------------------------------------------------|
  8. |     4   10jan2002        1   62    7     0          .             .          . |
  9. |     4   11jan2002        1   58    1     0         -6    -.85714286          1 |
 10. |     4   13jan2002        1   55    4     0          3             3          0 |
     +----------------------------------------------------------------------------------+
```

4.3 Comparing the by, tsset, xtset commands

Another way to find differences within vectors, we can use the tsset or xtset command to establish the times series (tsset) or panel data (xtset). We can use our bysort with subscripting/indexing.

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use tv1, clear
bysort kidid (dt): generate ltv = tv[_n-1]
list, sepby(kidid)
```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell


(4 missing values generated)

     +--------------------------------------------------+
     | kidid          dt   female   wt   tv   vac   ltv |
     |--------------------------------------------------|
  1. |     1   07jan2002        1   53    1     1     . |
  2. |     1   08jan2002        1   55    3     1     1 |
     |--------------------------------------------------|
  3. |     2   16jan2002        1   58    8     1     . |
     |--------------------------------------------------|
  4. |     3   18jan2002        0   60    2     0     . |
  5. |     3   19jan2002        0   63    5     1     2 |
  6. |     3   21jan2002        0   66    1     1     5 |
  7. |     3   22jan2002        0   64    6     0     1 |
     |--------------------------------------------------|
  8. |     4   10jan2002        1   62    7     0     . |
  9. |     4   11jan2002        1   58    1     0     7 |
 10. |     4   13jan2002        1   55    4     0     1 |
     +--------------------------------------------------+
```

tsset

Instead we can establish a time series with *tsset*

We'll need to specify that our cross-sectional groups is kidid. We'll need to specify our date variable with dt. We'll need to use the option, daily, to specify that time period is daily as opposed to weeks, months, years. Or we can specify delta(1) for one day.

We can use the operator *L.var* to specify that we want to a lag of 1 day

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use tv1, clear
sort kidid dt
tsset kidid dt, daily delta(1)
generate lagtv = L.tv
```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell

       panel variable:  kidid (unbalanced)
        time variable:  dt, 07jan2002 to 22jan2002, but with gaps
                delta:  1 day

(6 missing values generated)
```

We can use the operator F.var to specify that we want a lead of 1 day

```
generate leadtv = F.tv
list, sepby(kidid)
```

```
(6 missing values generated)

     +--------------------------------------------------------------+
     | kidid          dt   female   wt   tv   vac   lagtv   leadtv |
     |--------------------------------------------------------------|
  1. |     1   07jan2002        1   53    1     1       .        3 |
  2. |     1   08jan2002        1   55    3     1       1        . |
     |--------------------------------------------------------------|
  3. |     2   16jan2002        1   58    8     1       .        . |
     |--------------------------------------------------------------|
  4. |     3   18jan2002        0   60    2     0       .        5 |
  5. |     3   19jan2002        0   63    5     1       2        . |
  6. |     3   21jan2002        0   66    1     1       .        6 |
  7. |     3   22jan2002        0   64    6     0       1        . |
     |--------------------------------------------------------------|
  8. |     4   10jan2002        1   62    7     0       .        1 |
  9. |     4   11jan2002        1   58    1     0       7        . |
 10. |     4   13jan2002        1   55    4     0       .        . |
     +--------------------------------------------------------------+
```

xtset

We can establish a panel series with *xtset*

We'll need to specify that our cross-sectional group is kidid. We'll need to specify our time period is dt. We'll use a delta of 1 to specify that the differnce is 1 day. Or, we can use *daily*, as well

Generate a lag with the l.var operator and generate a lead with the f.var

```
cd "/Users/Sam/Desktop/Econ 645/Data/Mitchell"
use tv1, clear
sort kidid dt
xtset kidid dt, daily delta(1)
generate lagtv = l.tv
generate leadtv = f.tv
```

```
/Users/Sam/Desktop/Econ 645/Data/Mitchell

       panel variable:  kidid (unbalanced)
        time variable:  dt, 07jan2002 to 22jan2002, but with gaps
                delta:  1 day

(6 missing values generated)

(6 missing values generated)
```

What do you notice? You can see that there are some leads and lags missing. Why? Because there is an unbalance panel and the daily differences cannot be computed if we are missing days. In this case, we can use the bysort with subscripting indexing

```
list, sepby(kidid)
bysort kidid (dt): gen bylagtv=tv[_n-1]
bysort kidid (dt): gen byleadtv=tv[_n+1]
list, sepby(kidid)
```

```
     | kidid          dt   female   wt   tv   vac   lagtv   leadtv |
     |------------------------------------------------------------|
  1. |     1   07jan2002        1   53    1     1       .        3 |
  2. |     1   08jan2002        1   55    3     1       1        . |
     |------------------------------------------------------------|
  3. |     2   16jan2002        1   58    8     1       .        . |
     |------------------------------------------------------------|
  4. |     3   18jan2002        0   60    2     0       .        5 |
```

```
  5. |     3   19jan2002        0   63    5     1       2         . |
  6. |     3   21jan2002        0   66    1     1       .         6 |
  7. |     3   22jan2002        0   64    6     0       1         . |
     |--------------------------------------------------------------|
  8. |     4   10jan2002        1   62    7     0       .         1 |
  9. |     4   11jan2002        1   58    1     0       7         . |
 10. |     4   13jan2002        1   55    4     0       .         . |
     +--------------------------------------------------------------+

(4 missing values generated)

(4 missing values generated)

     +-----------------------------------------------------------------------------------+
     | kidid          dt   female   wt   tv   vac   lagtv   leadtv   bylagtv   byleadtv |
     |-----------------------------------------------------------------------------------|
  1. |     1   07jan2002        1   53    1     1       .        3         .          3 |
  2. |     1   08jan2002        1   55    3     1       1        .         1          . |
     |-----------------------------------------------------------------------------------|
  3. |     2   16jan2002        1   58    8     1       .        .         .          . |
     |-----------------------------------------------------------------------------------|
  4. |     3   18jan2002        0   60    2     0       .        5         .          5 |
  5. |     3   19jan2002        0   63    5     1       2        .         2          1 |
  6. |     3   21jan2002        0   66    1     1       .        6         5          6 |
  7. |     3   22jan2002        0   64    6     0       1        .         1          . |
     |-----------------------------------------------------------------------------------|
  8. |     4   10jan2002        1   62    7     0       .        1         .          1 |
  9. |     4   11jan2002        1   58    1     0       7        .         7          4 |
 10. |     4   13jan2002        1   55    4     0       .        .         1          . |
     +-----------------------------------------------------------------------------------+
```

What is the difference? xtset vs tsset

From Nick Cox: xtset allows a panel identifier only. tsset allows a time identifier only. Where they overlap is when two variables are supplied in which case the first is treated as a panel identifier and the second as a time variable.

4.4 Exercises

Let's grab the CPS and generate subgroup analysis. We will be using unweighted data for simplicity 1. What are the average wages by sex? 2. What are average wages by state 3. What are the median wages by sex 4. What are the median wages by state 5. What is the 75th percentile of wages by race? 6. What is the 25th percentile of wages by marital status?

```
use "/Users/Sam/Desktop/Econ 645/Data/CPS/jun23pub.dta",replace
```

Use bysort state: egen totalvar=total(var1)

Mitchell, N.M. (2020). *Data Management Using Stata: A Practical Handbook.* (2nd ed.). Stata Press.

UCLA Advanced Research Computing. (2025). *How can I access information stored after I run a command in Stata.* Retrieved from https://stats.oarc.ucla.edu/stata/faq/how-can-i-access-information-stored-after-i-run-a-command-in-stata-returned-results/

Wooldridge (2009). *Introductory Econometrics: A Modern Approach.* (4th ed.). South-Western Cengage Learning.